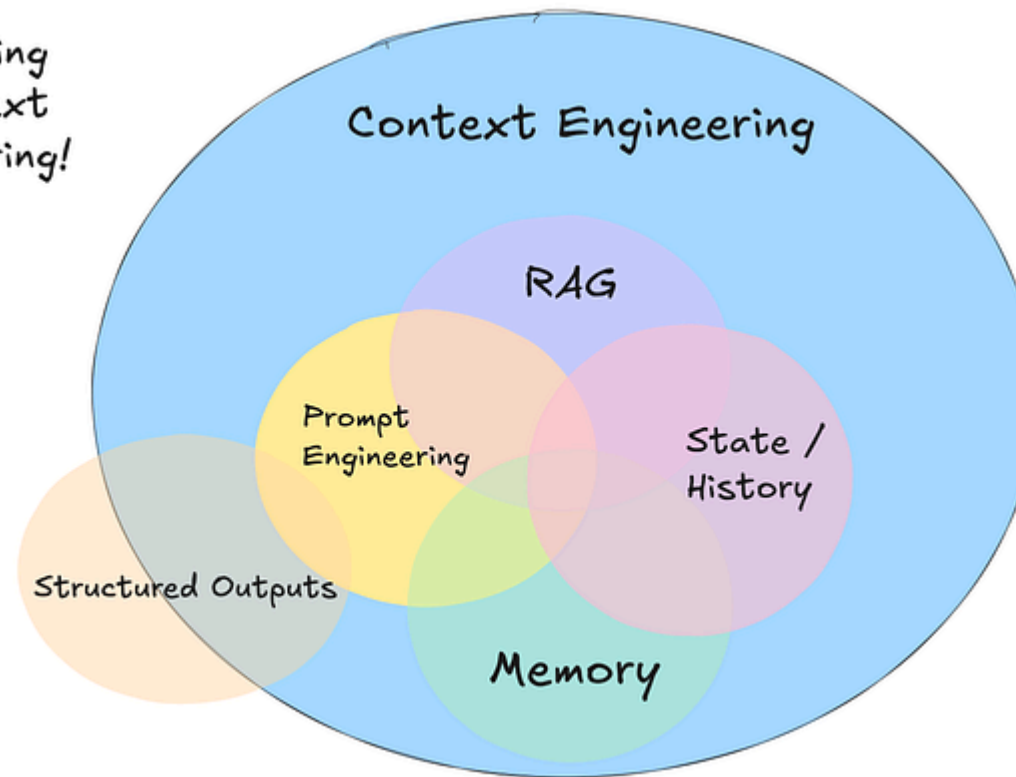


< Go to the original

Everything
is Context
Engineering!



Context Engineering Over Prompt Engineering

Prompt Engineering is Dead, it's time for Context Engineering.

Vishal Rajput

 AIGuys a11y-light ~12 min read · August 6, 2025 (Updated: August 6, 2025) · Free: No

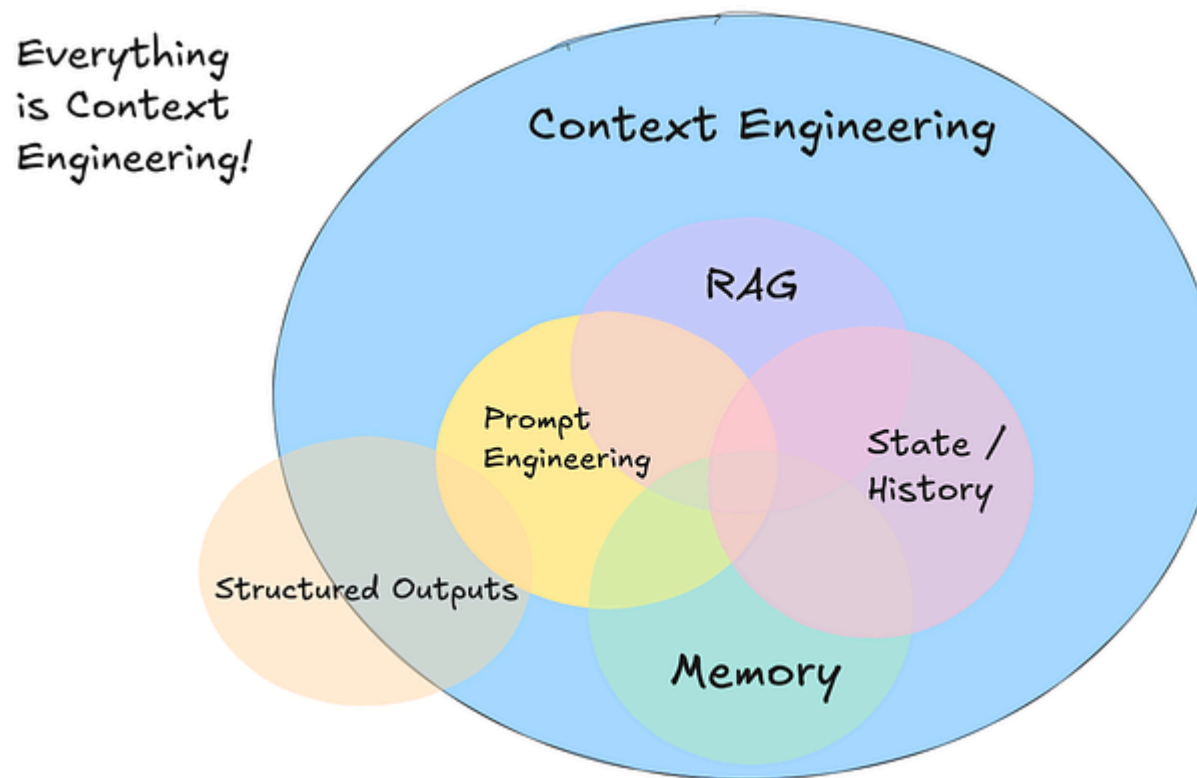
Prompt Engineering is Dead, it's time for Context Engineering.

Do you remember a year or so ago, when companies were offering up to 300K USD for prompt engineering? We don't see these posts or the company anymore. Prompt engineering was kind of hacky from the start. It was the fancy way to make everyone believe that they were engineers. But now things have changed and have taken the approach of traditional software engineering with the power of LLMs. So, let's look at the ultimate guide on how to build real agentic workflows that actually scale and don't break in production.

Table Of Contents

- Context Is Everything, Not Prompting
- Natural Language → Tool Calls
- Own Your Prompts
- Own Your Context Window
- Tools Are Structured Outputs

- Launch, Pause, Resume. Lifecycle Management via Simple APIs
- Own Your Control Flow
- Compact Errors into Context Window
- Next Gen Agents: Modular, Accessible & Stateless Agents



<https://github.com/humanlayer/12-factor-agents/blob/main/content/factor-03-own-your-context-window.md>

Leave Agentic AI Frameworks And Build Agents From Scratch

I'll be honest with you, I hate most agent-based AI workflows; they are simply unusable in the real world at scale...

medium.com

Context Is Everything, Not Prompting

Despite my distaste towards prompt engineering from the beginning, I can't deny all the past work done in prompt engineering. There is no one-size-fits-all type of prompt technique, but people have shown time and time again better performance on specific datasets with specific prompting techniques.

Knowing that with certain prompting techniques, we can get better responses from our language models, we should definitely try to understand the entire landscape of prompting and use proper prompting techniques.

But prompting alone is not enough; it is nothing but a small chunk of Context Engineering.

The Prompt Report: Prompt Engineering Techniques

Prompting Techniques Survey

medium.com

Software began as directed graphs, resembling flowcharts — a natural representation of sequential or branching logic.

And then came tools like Apache Airflow, Prefect, Dagster, and others formalized workflows as DAGs, enabling monitoring, retries, modularity, and observability.

But as the field of machine learning evolved, ML models became practical, DAGs started embedding model steps (e.g. classification, summarization), but remained deterministic: LLMs were just another step in a static pipeline.

The Promise of Agents

- This means less code, better error recovery, and more creative or adaptive behavior from agents.

Agents as Loops (Micro-agent pattern):

1. LLM emits a JSON tool call (next action).
2. Deterministic code executes it.
3. Append result to context and repeat until completion.

This replaces fixed DAG execution with flexible, iterative workflows.

Why the Loop Pattern Often Fails?

As context grows, agents tend to drift, repeating actions or losing coherence. Long context windows (10-20 turns) degrade reliability substantially — even high-quality LLMs struggle in those regimes. Many papers have shown that LLMs are not very reliable after a 32k context window, even if they can ingest a million tokens. The more context there is, the more chances there are for the LLM to hallucinate.

What Actually Works: Micro-Agents

- Micro-agents handle narrow, focused tasks (typically 3–10 steps), minimizing context drift.
- This pattern enables controlled human feedback loops without runaway context problems.

"DeployBot" Example workflow:

- Human merges a PR → deterministic CI deploys staging tests → an agent proposes deploy to production
- The agent triggers human approval steps (e.g. deploy backend first, then frontend)
- After approvals and executions, e2e tests on production run, agent signals completion or rolls back on failure ([GitHub](#)).
- Value lies in using LLM to interpret human feedback, not to manage full state or logic.

Software first: agents are software, not magic.

Define goals and permissible transitions, not all paths.

Contain LLM context: small, focused agents maintain coherence.

If you want more convincing arguments about why you should leave agentic frameworks:

Leave Agentic AI Frameworks And Build Agents From Scratch

I'll be honest with you, I hate most agent-based AI workflows; they are simply unusable in the real world at scale...

medium.com

Natural Language → Tool Calls

Software is all about the controlled flow of information; using free-form LLMs is not very helpful (in most cases). What we want is structured and controlled output.

Structured outputs prevent ambiguity and execution errors by binding intent to well-defined operations. It also supports deterministic programming execution, making agent behavior predictable and traceable

- Avoid multi-step or vague instructions; strive for single-intent conversions per call

If the model is uncertain about the intent (below threshold), escalate to human intervention rather than risk wrong execution

✓ TL;DR

Principle	Description
Atomicity	One intent → One tool call
Structure	Output must follow a strict schema (e.g. JSON)
Reliability	Use validation and confidence thresholds; escalate if needed

User Input: "Show me tomorrow's meetings."

Agent Output:

```
{ "action": "get_calendar_events", "date": "2025-08-05" }
```

👉 Own Your Prompts

Always **retain full control over your prompt contents** — treat prompts as first-class software artifacts, not opaque abstractions or framework-managed black boxes.

✅ Good vs Bad Practices

✅ Good Practice

Prompt templates saved in code, version-controlled, and tested

Direct editing and tuning of agent prompts

Owned, reviewed, iterated prompt definitions

❌ Common Pitfall

Sloppy prompt text hardcoded or hidden deep in framework

Relying entirely on framework abstractions for prompt generation

Using black-box templates you can't inspect or debug

Example of a prompt-as-function pattern:

Copy

```
function determineNextStep(thread: string): NextStep {
  prompt = `
    You are a helpful assistant that manages deployments.
    Conversation:
    ${thread}
  `
```

Factor 2 - Own your prompts

```
Agent:
  role: str
  goal: str
  personality: str

Task:
  instructions: str
  expected_output: Type[BaseModel]
  tools: list[tool]

agent.run(task)
```

black box

I don't know what's better, but I know you want to be able to try EVERYTHING



```
function DetermineNextStep(
  // to keep this clean, make the client turn the thread into a prompt-ready string,
  // didn't wanna solve that in jinja (although long term that's probably the best solution)
  thread: string
) -> ClarificationRequest | CreateIssue | ListTeams | ListIssues | ListUsers | DoneForNow | AddComment |
  client CustomGPT4o

prompt #
{{ _role("system") }}

You are a helpful assistant that helps the user with their linear issue management.
You work hard for whoever sent the inbound initial email, and want to do your best
to help them do their job by carrying out tasks against the linear api.

Before creating an issue, you should ensure you have accurate team/user/project ids.
You can list_teams and list_users and list_projects functions to get ids.

If you are BCC'd on a thread, assume that the user is asking you to look up the related issue and

Always think about what to do first, like:

- ...
- ...
- ...

{{ _role("user") }}
```

full control

Benefits: Flexibility, Optimization

<https://github.com/humanlayer/12-factor-agents/blob/main/content/factor-02-own-your-prompts.md>

Key benefits of owning your prompts:

1. **Full Control:** Write exactly the instructions your agent needs, no black box abstractions

3. Iteration: Quickly modify prompts based on real-world performance
4. **Transparency:** Know exactly what instructions your agent is working with
5. **Role Hacking:** take advantage of APIs that support nonstandard usage of user/assistant roles — for example, the now-deprecated non-chat flavor of OpenAI "completions" API. This includes some so-called "model gaslighting" techniques

Note: Your prompts are the primary interface between your application logic and the LLM.

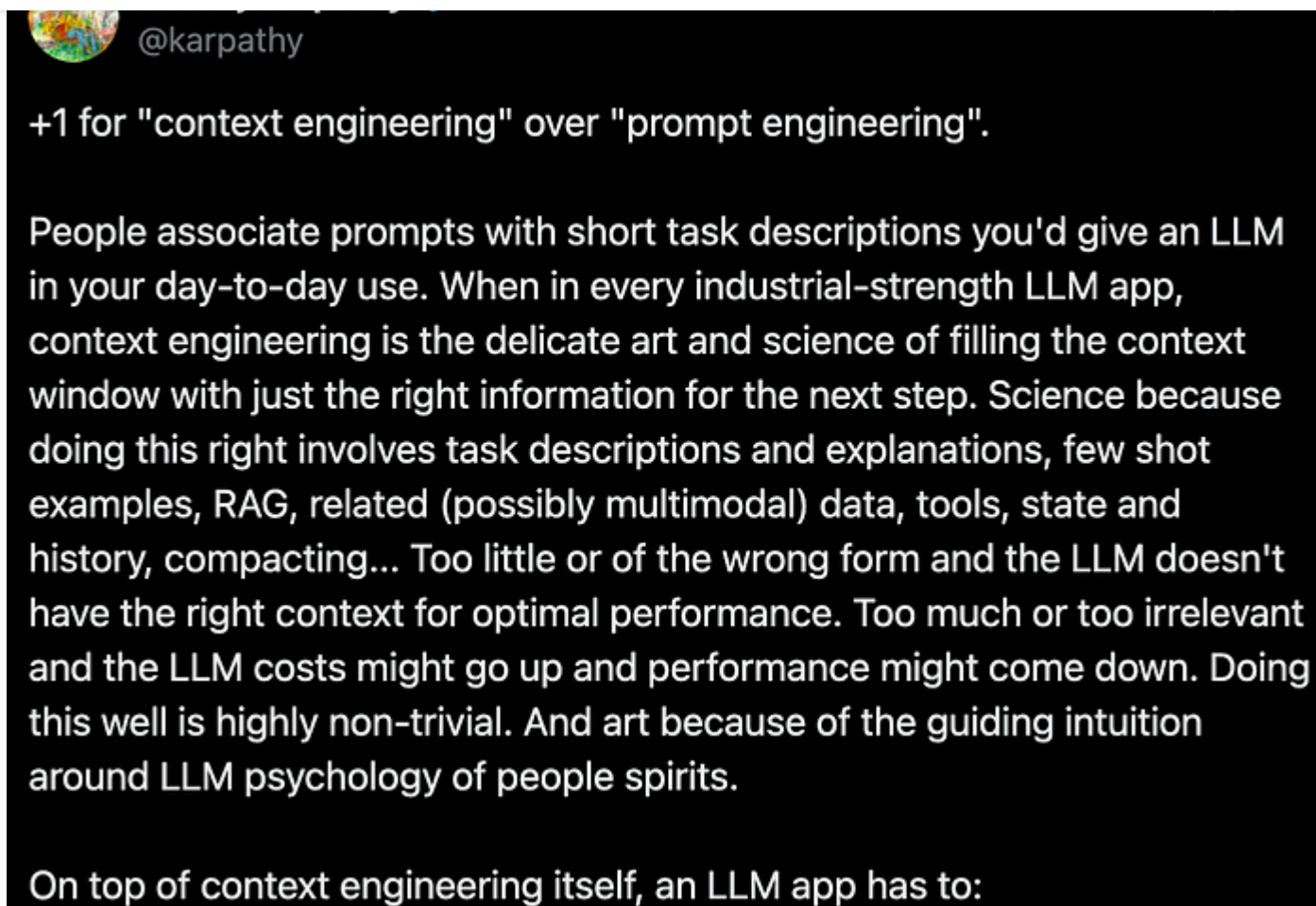
Having full control over your prompts gives you the flexibility and prompt control you need for production-grade agents.

I don't know what's the best prompt, but I know you want the flexibility to be able to try **EVERYTHING**.



Own Your Context Window

I will keep mentioning again and again, **CONTROL** is everything for software.



You should always listen to Karpathy

Design your context intentionally, not passively through chat logs or one-size-fits-all messaging formats.

Factor 3 Principle	Summary
You own the context window	Handcraft exactly what the LLM sees
Structure is key	Use compact, semantic formats
Relevance over verbosity	Only pass what matters now
Errors go into context	So agents can self-heal or escalate
Runtime flexibility	Schema evolves with use case

Why It Matters

- **Information Efficiency:** Structured formats like XML/YAML pack more meaning into fewer tokens, maximizing scarce context capacity.
- **Error Recovery:** Logging tool outputs and error states inside the context enables the model to reason about what went wrong and retry intelligently.
- **Privacy & Safety:** Exclude irrelevant or sensitive data proactively — for risk reduction and compliance.
- **Format Flexibility:** Adapt the schema as your system evolves; restructure or prune entries to suit runtime needs.

Key benefits of owning your context window:

1. **Information Density:** Structure information in ways that maximize the LLM's understanding
2. **Error Handling:** Include error information in a format that helps the LLM recover. Consider hiding errors and failed calls from the context window once they are resolved.
3. **Safety:** Control what information gets passed to the LLM, filtering out sensitive data
4. **Flexibility:** Adapt the format as you learn what works best for your use case
5. **Token Efficiency:** Optimize context format for token efficiency and LLM understanding

Tools Are Structured Outputs

Treat *tool calls* not as black-box API proxies but as **explicit, structured decision documents** — typically JSON — with clearly defined fields. These outputs are the interface between LLM reasoning and deterministic software that actually performs actions.

Define an explicit schema (e.g. via JSON Schema or typed dataclasses) that enforces key fields like `action_type` , `target` , optional metadata, and `confidence_score` . For example:

Copy

```
{
  "action": "deploy_service",
  "service": "backend",
  "version": "v1.2.3",
  "requires_approval": true,
  "confidence": 0.92
}
```

✓ TL;DR

Principle

Summary

Structured outputs

Use typed JSON outputs instead of LLM "free text"

Typed & validated

Enforce schema compliance before running tool logic

Document decisions

Log every tool output for traceability, audit, or replayability

See [Prompting vs JSON Mode vs Function Calling vs Constrained Generation vs SAP](#), [When should I use function calling, structured outputs, or JSON mode?](#) and [OpenAI JSON vs Function Calling](#).



Unify Execution State and Business State

Even outside the AI world, many infrastructure systems try to separate "execution state" from "business state". For AI apps, this might involve complex abstractions to track things like current step, next step, waiting status, retry counts, etc. This separation creates complexity that may be worthwhile, but may be overkill for your use case.

As always, it's up to you to decide what's right for your application. But don't think you *have* to manage them separately.

More clearly:

- Execution state: current step, next step, waiting status, retry counts, etc.
- Business state: What's happened in the agent workflow so far (e.g. list of OpenAI messages, list of tool calls and results, etc.)



<https://github.com/humanlayer/12-factor-agents/blob/main/img/155-unify-state-animation.gif>

This approach has several benefits:

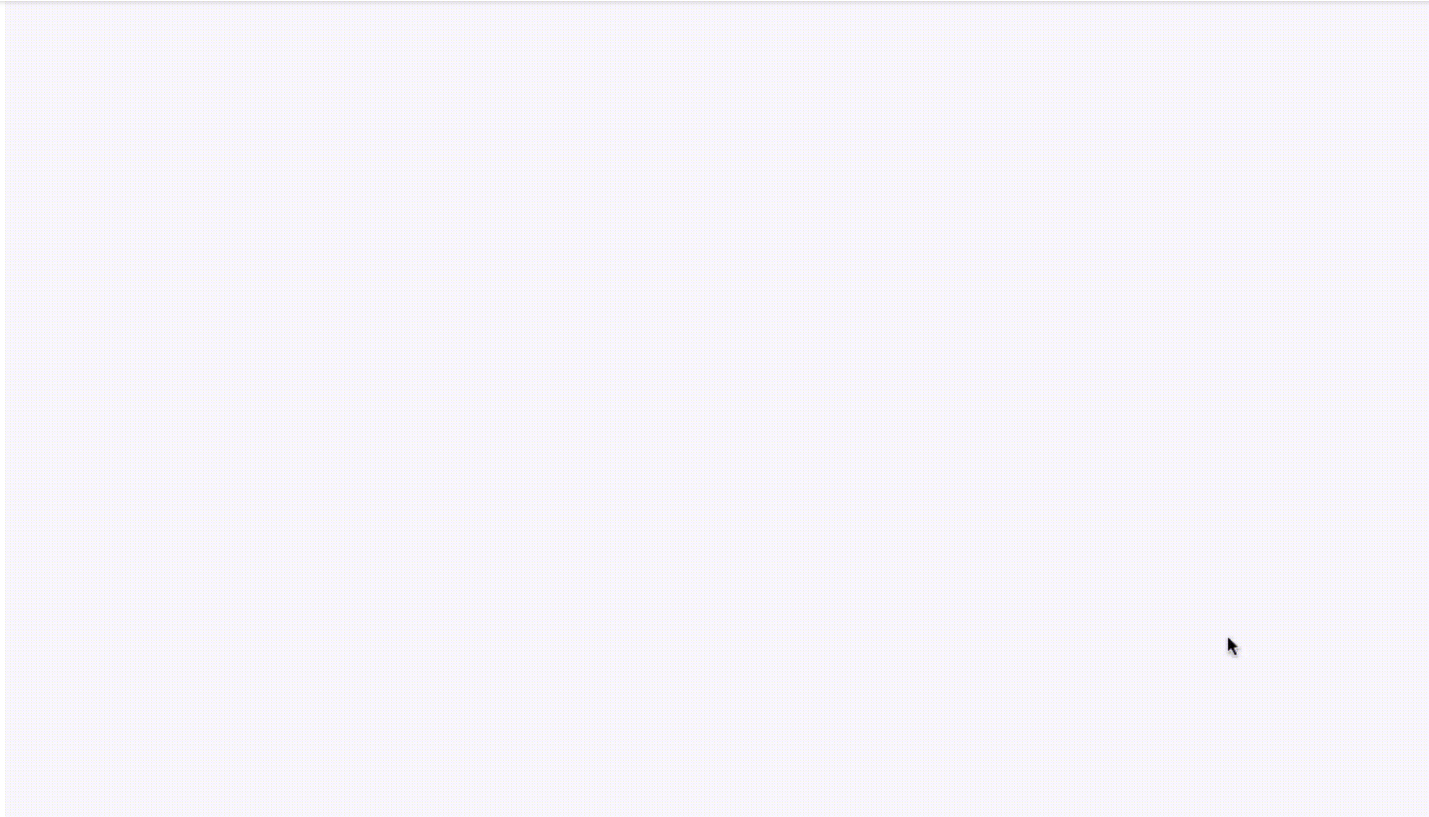
1. **Simplicity:** One source of truth for all state
2. **Serialization:** The thread is trivially serializable/deserializable
3. **Debugging:** The entire history is visible in one place
4. **Flexibility:** Easy to add new state by just adding new event types
5. **Recovery:** Can resume from any point by just loading the thread

7. Human Interfaces and Observability: Trivial to convert a thread into a human-readable markdown or a rich Web app UI

It should be easy for users, apps, pipelines, and other agents to launch an agent with a simple API.

Agents and their orchestrating deterministic code should be able to pause an agent when a long-running operation is needed.

External triggers like webhooks should enable agents to resume from where they left off without deep integration with the agent orchestrator.



<https://github.com/humanlayer/12-factor-agents/blob/main/img/165-pause-resume-animation.gif>

Launch, Pause, Resume: Lifecycle Management via Simple APIs

By default, LLM APIs rely on a fundamental HIGH-STAKES token choice: Are we returning plaintext content, or are we returning structured data?

content: "the weather in tokyo is 57 and rainy"

OR

content: "for what postal code?"

OR

tool_calls: [...]

model switches between
tools and plaintext

```
tool_calls: [{
  function: "final_answer",
  arguments: {
    message: "the weather in tokyo is 57 and rainy",
  }
}]
```

OR

```
tool_calls: [{
  function: "request_clarification",
  arguments: {
    message: "for what postal code?",
  }
}]
```

OR

```
tool_calls: {
  function: "check_weather_in_city",
  arguments: {
    "city": "tokyo",
    "postal_code": "12345"
  }
}
```

everything
is tools

Final answer vs. I need input vs. call tool is fiddly

Benefits: Clear instructions for the model, inner vs. outer loop,
access to multiple humans

<https://github.com/humanlayer/12-factor-agents/blob/main/content/factor-07-contact-humans-with-tools.md>

API Endpoint	Purpose
<code>/start</code>	Initialize a new agent workflow with its context and goal
<code>/pause</code>	Halt execution at a defined point for inspection or review
<code>/resume</code>	Continue from the last checkpoint using persisted state
<code>/status</code> (opt)	Query current step, confidence, or waiting state

Benefits:

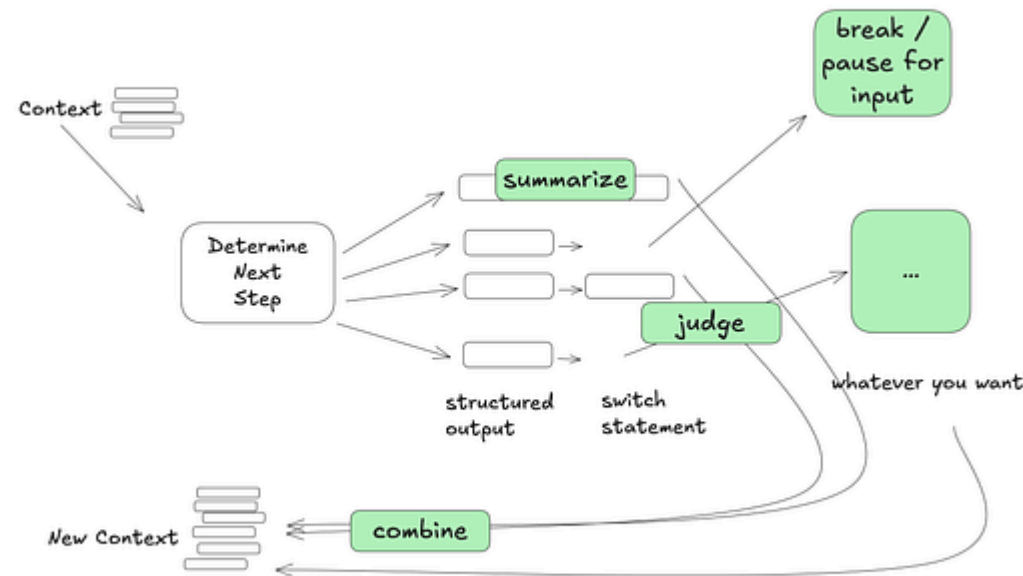
1. **Clear Instructions:** Tools for different types of human contact allow for more specificity from the LLM
2. **Inner vs Outer Loop:** Enables agents workflows outside of the traditional chatGPT-style interface, where the control flow and context initialization may be Agent→Human rather than Human→Agent (think, agents kicked off by a cron or an event)
3. **Multiple Human Access:** Can easily track and coordinate input from different humans through structured events
4. **Multi-Agent:** Simple abstraction can be easily extended to support Agent→Agent requests and responses

 Own Your Control Flow

If you own your control flow, you can do lots of fun things. Full control over the *agent's main think-act loop*: you decide when it iterates, pauses, escalates, or stops. Don't offload workflow sequencing to the LLM; keep logic in your deterministic code.

 TL;DR

Principle	Description
Control equals code	You write the loop and decision logic, not the LLM
Pause points are deterministic	Every stop / continue is defined in code, not inferred
Flow supports governance	Risk, human review, retries are enforceable steps
Flow enables auditability	You can trace and test each branch or handoff



Benefits: Interrupts, Process Control

<https://github.com/humanlayer/12-factor-agents/blob/main/content/factor-08-own-your-control-flow.md>

Build your own control structures that make sense for your specific use case. Specifically, certain types of tool calls may be a reason to break out of the loop and wait for a response from a human or another long-running task like a training pipeline. You may also want to incorporate a custom implementation of:

- summarization or caching of tool call results
- LLM-as-judge on structured output

- logging, tracing, and metrics
- client-side rate limiting
- durable sleep / pause / "wait for event"

Use a Structured Loop (Pseudo-code example)

Copy

```
while not done:
    decision = determine_next_step(prompt_from(thread))
    if decision.intent == "request_clarification":
        send_human_contact(decision)
        save_state(thread)
        break # pause until human responds
    elif decision.intent == "fetch_data":
        result = call_tool(decision)
        thread.events.append({"type": decision.intent + "_result", "data": result})
        continue # proceed to next loop iteration
    elif decision.intent == "trigger_action":
        execute_action(decision)
    done = True
```

This allows controlled branching between immediate tool execution, pause for human input, or termination.

between the moment of tool selection and the moment of tool invocation.

Without this level of resumability/granularity, there's no way to review/approve the tool call before it runs, which means you're forced to either:

1. Pause the task in memory while waiting for the long-running thing to complete (think `while...sleep`) and restart it from the beginning if the process is interrupted
2. Restrict the agent to only low-stakes, low-risk calls like research and summarization
3. Give the agent access to do bigger, more useful things, and just yolo hope it doesn't screw up

Compact Errors into Context Window

This one is a little short but is worth mentioning. One of these benefits of agents is "self-healing" — for short tasks, an LLM might call a tool that fails. Good LLMs have a fairly good chance of reading an error message or stack trace and figuring out what to change in a subsequent tool call.

Copy

```
while True:
    next_step = await determine_next_step(thread_to_prompt(thread))
    thread["events"].append({
        "type": next_step.intent,
        "data": next_step,
    })
    try:
        result = await handle_next_step(thread, next_step) # our switch statement
    except Exception as e:
        # if we get an error, we can add it to the context window and try again
        thread["events"].append({
            "type": 'error',
            "data": format_error(e),
        })
        # loop, or do whatever else here to try to recover
```

You may want to implement an errorCounter for a specific tool call, to limit to ~3 attempts of a single tool, or whatever other logic makes sense for your use case.

Benefits:

1. **Self-Healing:** The LLM can read the error message and figure out what to change in a subsequent tool call

I'm sure you will find that if you do this TOO much, your agent will start to spin out and might repeat the same error over and over again.

Next Gen Agents: Modular, Accessible & Stateless Agents

Agents should be domain-specific and limited in scope (typically 3–20 steps), avoiding monolithic workflows.

Why it matters:

- Reduces context-window drift.
- Improves reliability, testability, debugging, and reasoning clarity.
- Supports composability — teams can evolve or extend agents independently.

Agents should be activated by any relevant trigger — Slack, email, SMS, webhooks, cron, or other events — and respond via the same channel.

Why it matters:

- Enhances accessibility and adoption — agents become digital coworkers.

- Enables high-stakes operations by integrating user oversight into natural workflows.

Design agents as pure functions: given **input state** (from context/history) and **trigger**, produce **new state + a tool-call/output**. The agent itself holds no internal state across runs.

Why it matters:

- Facilitates deterministic behavior — given same inputs, agent outputs are reproducible.
- Enables horizontal scaling: any instance can process any request.
- Simplifies testing, debugging, replayability, and auditing.

Small, single-purpose agents

High reliability, easy to test and reason

Universal triggers

Accessible from any interface or event source

Stateless, functional agent

Scalable, reproducible, auditable

Combined Architecture Pattern

1. **Input:** An event or user message triggers an agent.
2. **Reducer function:** Reads current state + trigger → outputs structured decision and updated state.
3. **Deterministic orchestration:** Your code dispatches tool calls, logs events, and applies control flow.
4. **Agent finishes quickly**, passing the state forward to the next micro-agent if needed.
5. **Human-in-loop escalation** when needed via applications.
6. **Next agent** invoked when appropriate, using the same pattern.

[agents/tree/main](#)

Writing articles like this takes considerable effort and time. Please subscribe and follow me if my content adds any value to you.

Newsletter: <https://medium.com/aiguys/newsletter>

X: <https://x.com/RealAIGuys>

Also, looking to solve a challenge for business-related documents, check out:
<https://number7ai.com/>

[#ai-agent](#) [#agentic-ai](#) [#agentic-ai-framework](#) [#artificial-intelligence](#) [#ai](#)